

Evaluating the Performance of Sidecar-based and Sidecarless Cloud-native Service Mesh Solutions

Karumuri Meher Hasanth and Deepak Nadig

Dept. of Computer and Information Technology, Purdue University, West Lafayette, IN 47907

Email: {kmeherha, nadig}@purdue.edu

Abstract—The software development landscape is rapidly evolving, marked by the shift from monolithic applications to microservices architectures and the adoption of agile methodologies. While developing microservices has become more effortless, the challenges of maintaining and deploying them persist, specifically with managing the complex dependencies associated with multiple microservice deployments. With an increasing demand for container orchestrators and Kubernetes emerging as the industry standard, networking microservices remains a key challenge. Service mesh, a solution to this problem, consists of a control and a data plane, with the former responsible for managing the components of the latter. Data planes in most service meshes are implemented using sidecar proxies, posing specific challenges when injected into application pods. A potential solution to these challenges lies in the sidecarless approach. This paper aims to explore the eBPF-based service mesh, Cilium, which eliminates the need for a sidecar while encompassing most service mesh capabilities. Further, we benchmark Cilium's performance, analyze its effectiveness, and compare its performance with Istio and Linkerd. Our evaluations show that Linkerd outperforms other solutions with respect to response times, with Linkerd performing 29.85% and 63.43% better than Cilium and Istio, respectively.

Index Terms—Service Mesh, Cilium, eBPF, Sidecarless.

I. INTRODUCTION

Before the rise of microservices, legacy software often relied on monolithic architectures, impeding the transition to agile development methods. Microservices emerged as a solution, comprising small, independent units of code that execute specific business functions and interact through APIs. This migration from monolithic to microservices allowed for the segmentation of applications into manageable components, which aligns with agile principles. The adoption of microservices has been further facilitated by containerization and orchestration tools, with Kubernetes standing out as the primary solution for effective container management.

The advent of microservices has increased the complexity of traditional debugging methods and led to a growing need for advanced functionalities like service discovery, load balancing, and security. Network service mesh (NSM) addresses these requirements by separating networking logic from applications using proxies. Various open-source service mesh solutions have emerged to cater to different use cases. For example, Istio [1] utilizes Envoy as a sidecar proxy, providing encryption and distributed tracing features without additional software agents. Built with Rust, Linkerd [2] focuses on minimizing resource consumption compared to Istio. Consul [3] stands

out for its robust service discovery capabilities and ability to function within and outside Kubernetes. Traefik Mesh [4] targets efficient load balancing. Although these service meshes employ distinct strategies, they all share a common principle: offloading networking responsibilities to a sidecar proxy. However, Cilium [5] takes a different approach by using extended Berkeley packet filters (eBPF), eliminating the requirement for a sidecar proxy. Nevertheless, architectures reliant on service meshes and sidecars in service mesh solutions can increase resource usage, potentially impacting application performance.

Further, the rapid adoption of microservices architecture, fueled by cloud computing, has led to extensive efforts focusing on decomposing large monolithic applications into smaller, more manageable microservices. Studies comparing microservices to monoliths have revealed challenges such as managing interactions, dealing with inter-service communication overhead, and addressing data management issues. The distributed nature of microservices has introduced problems like network failures, service discovery, load balancing, and distributed tracing. Additionally, there are concerns about security, initial development overheads, monitoring, and advanced traffic management for inter-service communication.

In this paper, we evaluate the benefits of sidecarless service meshes such as Cilium. We study Cilium's architecture and its utilization of eBPF to eliminate the need for a sidecar. Through this analysis, we aim to comprehensively understand Cilium as both a service mesh and a CNI. Thus, this paper provides valuable insights into how Cilium operates as a container network interface (CNI) and a service mesh solution. To evaluate its capabilities, we deploy a microservices application [6] that employs the Cilium service mesh and assess its performance using an L7 traffic generator. Furthermore, we conduct a comparative analysis of its performance against two prominent service mesh solutions, namely Istio and Linkerd.

This paper is organized as follows: In Section II, we introduce the network service mesh, its features, and capabilities and outline the related works. In Section III, we detail Cilium's internal architecture and its operation. Section IV presents a qualitative comparison of Istio, Linkerd, and Cilium. In Section V, we describe our evaluation framework and experimental setup. In Section VI, we present a comprehensive performance analysis of three service solutions and discuss their capabilities and limitations. Further, we present the suitability of each service mesh solution for specific use cases in terms of its performance, ease of use, resource, and operational costs.

Lastly, in Section VII, we conclude our work and provide future directions for this work.

II. BACKGROUND AND RELATED WORK

This section provides a brief background on network service meshes, their features and capabilities, and the related works in the context of benchmarking service mesh implementations.

A. Network Service Mesh (NSM)

A network service mesh (NSM), or service mesh, is an infrastructure layer designed to manage service-to-service communication within microservices architectures, focusing on secure, reliable, and observable interactions. It provides functionalities such as service discovery, load balancing, encryption, authorization, and observability without modifying the application code. This paper will examine three primary service mesh solutions, namely Istio, Linkerd, and Cilium. Istio and Linkerd will serve as key benchmarks, as these two solutions are adopted by 88% of the industry service mesh deployments based on market share data [7], with Istio and Linkerd accounting for 47% and 41% of production use, respectively.

B. Service Mesh Capabilities and Features

The key capabilities and features of service meshes include:

1) *Traffic Management*: Traffic management is a fundamental feature of a service mesh that provides load balancing, traffic routing, and traffic shifting capabilities. Load balancing ensures that incoming requests are evenly distributed among the available service instances, optimizing resource utilization and preventing overloads. Traffic routing allows for intelligent routing based on various criteria like service names, paths, and headers, enabling flexible traffic management. Traffic shifting is essential for controlled deployments, enabling canary releases and gradual rollouts of new versions to minimize the impact of potential issues on the entire system.

2) *Service Discovery*: Service discovery simplifies managing dynamically changing service infrastructures. Dynamic service registration automatically registers new services as they come online and removes them when they go offline. A service catalog maintains an up-to-date list of available services and their locations, simplifying service lookup. DNS resolution further aids service discovery by providing a convenient way for applications to locate services.

3) *Observability*: Observability features in a service mesh provide insights into the performance and behavior of microservices. Metrics collection records data on request/response times, error rates, and other performance indicators. Distributed tracing offers visibility into the flow of requests across multiple microservices, aiding in debugging and performance analysis. Centralized logging aggregates logs from all services, simplifying troubleshooting and auditing.

4) *Traffic Telemetry*: Traffic telemetry involves monitoring the health of services and collecting performance data. Service health checks periodically assess the health of services and divert traffic away from unhealthy instances. Metrics and analytics offer insights into the system's overall performance and health. Alerting capabilities provide notifications and alarms for detecting and responding to abnormal behavior or performance degradation.

5) *A/B Testing and Canary Releases*: A/B testing and Canary releases allow controlled testing of new features or versions. In a service mesh, traffic can be selectively directed to a test group, enabling the evaluation of changes or new versions in a production-like environment before full deployment, thus reducing risks and ensuring a smoother transition.

6) *Multi-Cluster and Multi-Cloud Support*: Service meshes support communication between microservices deployed across different clusters or cloud providers. This capability is essential for hybrid and multi-cloud deployments, where services may span various environments.

7) *Policy Enforcement*: Policy enforcement [8] in a service mesh involves defining and enforcing rules that govern the behavior of microservices, ensuring security, traffic management, access control, and compliance. This includes enforcing authentication and authorization, encrypting communications, managing traffic through rate limiting and retries, and implementing role-based access controls. Additionally, it ensures compliance through auditing, data residency, and operational policies like circuit breakers and load balancing. These policies are centrally managed by the control plane and enforced at runtime by the data plane, ensuring consistent and reliable service operations across the microservices architecture.

C. Related Work

The industry has rapidly embraced the microservices model, particularly with cloud computing facilitating faster adoption. The work by the authors in [9] shows how to systematically break down monolithic applications into small, independent microservices, establishing functional boundaries. The authors in [10] present a comparison of the performance of microservices and monoliths across different granularities, analyzing metrics such as CPU performance, memory usage, disk consumption, network performance, response time, and bandwidth estimation. Microservices can increase complexity, especially in managing interactions, ensuring data consistency, and coordinating deployments [11]. Inter-service communication overhead, typically through APIs or network calls, leads to increased latency and potential bottlenecks [12]. Data management becomes more complex, requiring system-level support beyond software engineering practices [13]. The distributed nature of microservices introduces issues like network failures, service discovery, and load balancing [14]. Security challenges have also been overlooked, with existing research showing gaps in addressing vulnerabilities in microservices architecture [15]. Additionally, the initial development overhead for microservices can delay time-to-market [16], and monitoring and observability become increasingly difficult as the number

of services grows [17]. Finally, advanced traffic management, including A/B testing and canary deployments, adds another layer of complexity. In contrast to the above, our work focuses on evaluating the benefits of various service mesh solutions and the impact of a sidecarless design on performance, ease of use, and efficiency.

III. CILIUM ARCHITECTURE AND EBPF

Cilium leverages eBPF to provide visibility and control over systems and applications, achieving granularity and efficiency typical of in-kernel observability techniques. These enhancements are implemented seamlessly without necessitating any changes to the application code. eBPF works effectively with modern containerized workloads and more traditional setups like virtual machines and standard Linux processes. However, this approach introduces challenges and opportunities in securing service-to-service communications. Traditional Linux network security methods, such as `iptables`, rely on IP addresses and TCP/UDP ports for filtering traffic, but these IP addresses often change in dynamic microservices environments. The transient nature of containers makes traditional `iptables` methods less effective in adapting to application deployments, resulting in scaling issues with access control lists (ACL) and load-balancing tables. Additionally, using protocol ports as a means of security differentiation is no longer practical, as these ports are used for a wide range of service communications in container orchestration systems.

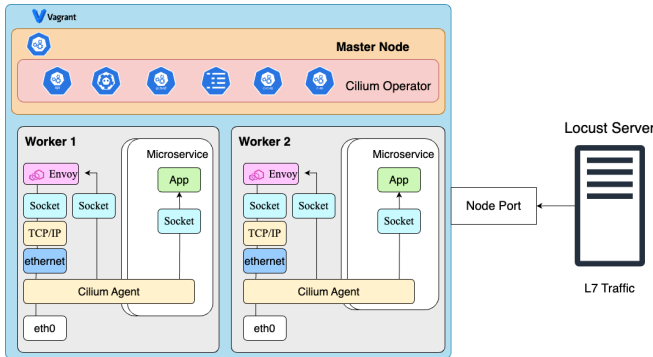


Fig. 1: Cilium Deployment Architecture

Cilium addresses these challenges using eBPF to provide security, visibility, and policy enforcement based on service, pod, or container identity instead of IP addresses. This approach allows filtering at the application layer, simplifying the application of security policy in highly dynamic environments by decoupling security from addressing. Further, it enhances security isolation by operating at the HTTP layer rather than traditional Layer 3 and Layer 4 segmentation.

Cilium integrates the service mesh layer directly into the kernel via eBPF, eliminating the need for sidecar proxies. This innovative approach effectively manages connectivity at both the networking and application protocol layers, handling protocols like IP, TCP, UDP, HTTP, Kafka, gRPC, and DNS with high efficiency. By utilizing eBPF, Cilium offers fast, scalable

networking that overcomes the performance limitations typically associated with traditional proxies, enabling direct and efficient communication between cloud-native services.

Additionally, Cilium provides a range of control plane options, allowing users to balance complexity and features according to their needs. These options include more straightforward choices like Ingress and Gateway API, as well as more comprehensive solutions like Istio and the full capabilities of Envoy accessed through the Envoy custom resource definition (CRD). Figure 1 shows a typical Cilium deployment with its internal architecture. The Cilium agent runs on each worker node and interacts with the deployed microservices through the agent with minimal overheads. The main components of the Cilium service mesh include Cilium components and Hubble, as detailed next.

A. Cilium Components

The following components form the core of the Cilium service mesh infrastructure.

1) *Agent*: The Cilium agent `cilium-agent` operates on every cluster node. Its core function is to receive configuration directives via Kubernetes or APIs, covering aspects such as network, service, and policy management, meeting monitoring requirements, and load-balancing workloads. The agent tracks container orchestration events from Kubernetes to identify container and workload states. The Cilium agent is responsible for managing eBPF programs in the kernel, which control container ingress and egress traffic.

2) *CLI*: The Cilium CLI client (`cilium`) is a command-line tool installed alongside the Cilium agent. It communicates with the REST API of the Cilium agent on the same node. The CLI enables users to inspect the state and status of the local agent and offers tools to access and validate the state of eBPF maps directly.

3) *Operator*: The Cilium operator is designed to manage cluster-wide tasks in a Kubernetes environment, centralizing functions that only need to be performed once for the entire cluster rather than on each node. This includes responsibilities like IP Address Management (IPAM) and maintaining state information in a key-value store (`kvstore`). The operator handles these tasks centrally and ensures consistency and efficiency across the cluster. Notably, the operator is not part of the critical path for real-time network traffic forwarding or policy enforcement. This allows the cluster to continue operating normally even if the operator is temporarily unavailable. However, the unavailability of the Cilium Operator can lead to specific issues depending on the cluster configuration. If the operator is required to allocate new IP addresses, its downtime can delay scheduling new workloads due to delays in IPAM. Additionally, the operator's role in updating the `kvstore` heartbeat key is crucial for maintaining the perceived health of the `kvstore`. If this update fails, Cilium agents might declare the `kvstore` unhealthy and restart, causing potential disruptions. Therefore, while not critical for immediate network operations, the operator's availability is essential for smooth and timely cluster management.

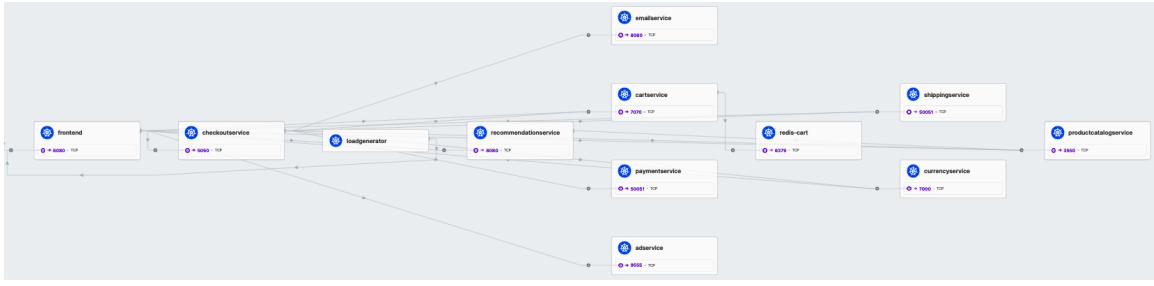


Fig. 2: Deployed Microservice Application Service Graph

4) *CNI Plugin*: The CNI plugin (`cilium-cni`) is responsible for scheduling pods and their lifecycle on each node. It communicates with the Cilium API on the node to initiate the required datapath configuration, ensuring the pod receives networking, load-balancing, and network policy support.

B. Hubble

Cilium uses Hubble for observability, and the following outlines how Hubble interacts with Cilium.

1) *Server*: The Hubble server operates on each node, collecting eBPF-based visibility data from Cilium. It is integrated into the Cilium agent to ensure high performance and minimal overhead. The server provides a gRPC service for retrieving flows and Prometheus metrics.

2) *Relay*: Relay (`hubble-relay`) is an independent component that recognizes all active Hubble servers and provides visibility into the cluster using the corresponding gRPC APIs. It provides a unified API that represents all the servers within the cluster.

3) *CLI and GUI*: - The Hubble CLI (`hubble`) is a command-line tool capable of connecting to the gRPC API of `hubble-relay` or the local server to retrieve flow events. The graphical user interface (Hubble-UI) leverages relay-based visibility to provide a graphical representation (See Figure 2) of service dependencies and application connectivity within the cluster.

IV. SERVICE MESHES: A QUALITATIVE COMPARISON

Compared to other service mesh architectures, Cilium incorporates a sidecar-less architecture, simplifying deployment. Key features of Cilium include service discovery, load balancing, traffic management, security, observability, policy enforcement, rate limiting, canary rollouts, and support for various protocols. Table I presents a comparative analysis of these features for three prominent service mesh solutions. Initially developed as a container network interface (CNI) using eBPF, Cilium enables efficient metric collection for the service mesh.

Istio leverages Envoy Proxy [18] to intercept service-to-service traffic and collect essential metrics. Each proxy instance consumes around 5 MB of memory, which can significantly increase resource usage as application complexity grows. In contrast to Istio and Linkerd, Cilium takes a unique approach by removing the need for a sidecar proxy altogether.

TABLE I: Comparison of Service Mesh Solutions

Feature	Linkerd	Istio	Cilium
Service Discovery	Yes	Yes	Yes
Traffic Management	Yes	Yes	Yes
Security	Yes	Yes	Yes
Observability	Yes	Yes	Yes
Policy Enforcement	Limited	Yes	Yes
Fault Injection	Limited	Yes	No
Circuit Breaking	Yes	Yes	No
Rate Limiting	Yes	Yes	Yes
Canary Rollouts	Yes	Yes	Limited
mTLS support	Yes	Yes	Yes

V. EXPERIMENTAL SETUP

Our experimental setup relies on high-performance bare metal nodes on the Chameleon Cloud testbed [19]. We deploy a Kubernetes cluster on bare metal nodes with dual-socket AMD EPYC 7763 processors with 128 cores, 256 threads, and 256 GiB of memory. A three-node cluster, including one master and two worker nodes, is deployed with Kubernetes version 1.24. We also deploy a complete microservices application (Google online botique [6]) comprising eleven inter-connected microservices. We expose the application frontend through a NodePort configuration and a public IP. We employ Locust [20] to generate traffic on several microservice application endpoints, including HTTP GET and POST requests, to evaluate the service meshes' robustness under various usage scenarios. Istio version v1.22.3 was deployed, followed by Linkerd v2.16, which employed the Calico v3.28 CNI. Lastly, Cilium v1.16.1 (also a CNI) was deployed, and the microservices were meshed using the service mesh infrastructure.

VI. RESULTS AND DISCUSSION

We rely on Locust to generate HTTP GET/POST traffic to the microservices' endpoints. Locust [20] is an open-source load-testing tool designed to test the performance of web applications and services under heavy load. It is user-friendly and scalable, with Python scripting capabilities for testing various web applications. Locust supports distributed testing, allowing us to simulate millions of simultaneous users with multiple machines. Further, it offers the flexibility to create complex test scenarios. Locust also provides various performance metrics, such as the number of requests per second, response times, and failure rates, aiding in analyzing application performance and

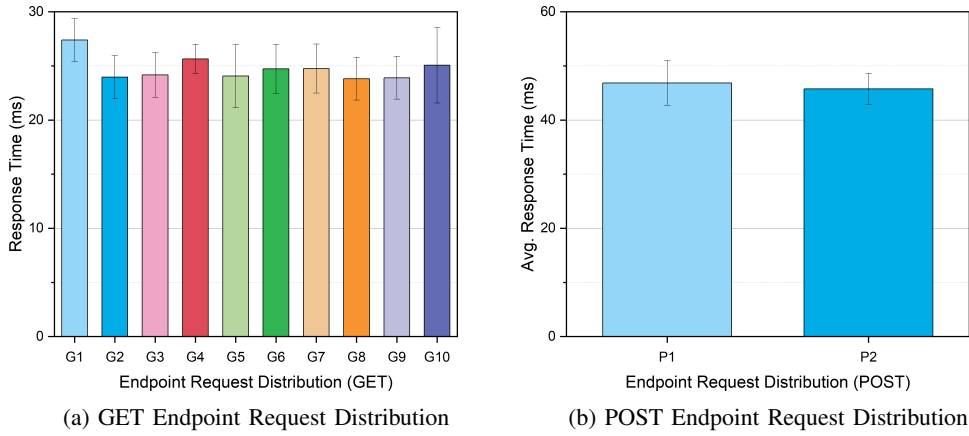


Fig. 3: Endpoint Request Distribution Statistics using Locust

robustness. The tool is customizable and extensible, allowing users to define the behavior of users (i.e., locusts) and extend functionality. Additionally, Locust is lightweight and efficient, using minimal resources compared to other tools, making it suitable for testing large-scale applications.

A. Endpoint Request Distribution

Using locust, we spin up 50 concurrent to generate endpoint traffic. This traffic was swarmed (a Locust term) to the exposed endpoints of the application's front end via a NodePort configuration. We then collect response times for ten minutes. Our deployed microservice application has a total of twelve (12) endpoints. Requests were distributed amongst these endpoints. Request distribution for each run is shown in Figure 3, with an average load of 574.21 requests. On average, 11 requests per second were sent to each endpoint. Figures 3a and 3b show the GET and POST endpoint distribution statistics for our deployed application with a 95% confidence interval computed from eight runs. On average, each `/GET` endpoint receives 20 – 30 requests, as shown in Figure 3a. Similarly, Figure 3b indicates the aggregate number of requests processed by the `/POST` endpoints, with approximately 45 POST requests per endpoint on average. From our analysis, on average, about 10.5%, 52.6%, 5.3%, 5.3%, 10.5%, and 15.8% requests were sent to `addToCart`, `browseProduct`, `checkout`, `index`, `setCurrency`, and `viewCart` services, respectively.

B. Latency Analysis

Figure 4a and Figure 4b illustrate the request completion time percentiles for GET and POST endpoints, respectively. From the figures, we note that 99% of requests for the Cilium service mesh were completed, on average, within 160 ms for the GET requests and 200 ms for the POST requests. This data was collected on the `/GET` CART endpoint deployed on the BareMetal node. We used the Cilium CNI for this evaluation, and we note that the response times increase exponentially from the 50th percentile to the 99th percentile. Further, we note that a few requests take about thrice the 99th percentile to complete for GET requests and about twice the 99th percentile for POST requests.

C. Response Time Comparison

Figure 4c illustrates the performance of three primary service mesh solutions under identical workloads and system conditions. We employed Locust to generate L7 application traffic and generate real-time load. Compared to Istio and Linkerd, Cilium consistently outperforms Istio with lower response time latencies, as shown in Figure 4c. Although Linkerd performs better in our tests, it lacks critical service mesh capabilities and is less usable than the other two solutions (See Section VI-D). Istio uses Envoy proxy as a sidecar and provides rich metrics with the highest adoption in production. Linkerd uses a lightweight rust-based proxy as a sidecar, and Cilium takes a unique sidecarless route. We measured the response time latencies for each service mesh for 25 users per test at a spawn rate of 2 users per second and a 10-minute runtime. We observe that Linkerd outperforms Istio and Cilium with consistently low response times, indicating better performance. Further, Istio has the highest response times of the three solutions. Cilium performs better than Istio in each case, with a response time higher than Linkerd.

D. Discussion

Our performance analysis shows that Linkerd is the best-performing service mesh among the three solutions. However, the suitability of a particular service mesh solution depends on additional factors, including CNI choice, resource overheads, and operational costs. A key consideration that impacts resource overheads and operations/maintenance costs is the choice of CNI. Linkerd and Istio cannot function independently without a CNI. While both support most mainstream CNIs, CNI deployments and maintenance represent an additional burden on the operator and the infrastructure. Cilium, on the other hand, serves as both CNI and service mesh. This simplifies deployments and maintenance and reduces overall resource utilization and operational overheads. As the CNI consumes valuable cluster resources in the form of `DaemonSets` and `Deployments`, Cilium eases administrative burdens as the operator no longer manages two different control plane components. Table II compares various factors

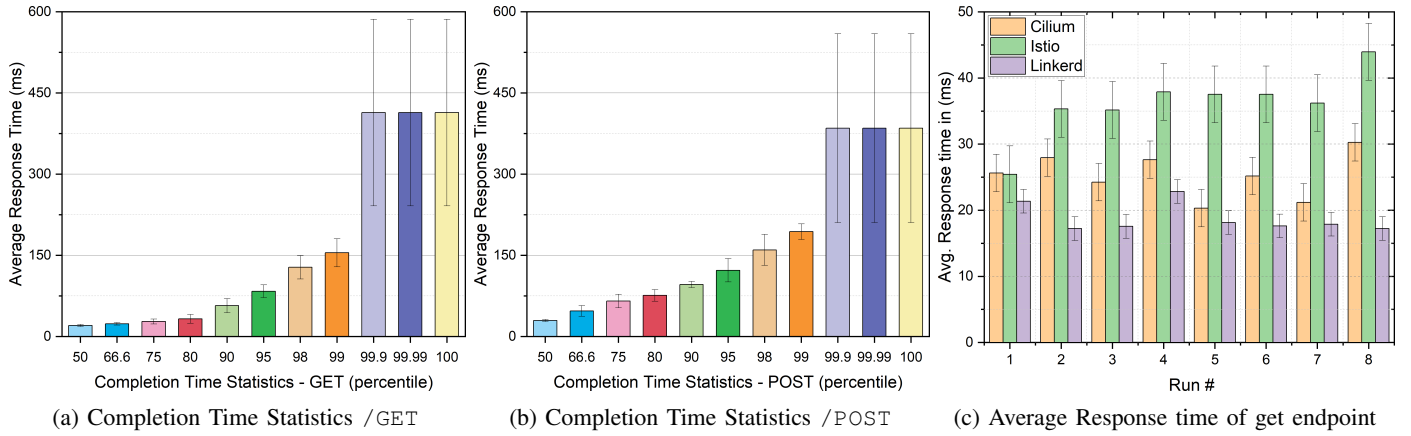


Fig. 4: Service Mesh Benchmarks - Istio, Linkerd, and Cilium.

that must be considered before choosing a service mesh solution.

TABLE II: Factors Impacting Service Mesh Choice

	Linkerd	Cilium	Istio
Latency Performance	Best	Better	Good
Ease of Use and Maintenance	Moderate	Easy	Difficult
Overall Resource Utilization	Moderate	Low	High
Operational Costs (OPEX)	Moderate	Low	High

VII. CONCLUSION AND FUTURE WORK

In this paper, we present a comprehensive performance analysis of three service mesh solutions, including Istio, Linkerd, and Cilium. We present a comprehensive qualitative analysis of their features and capabilities. Our testbed evaluations show that Linkerd outperforms the other two with the lowest average response times. We note that Linkerd performs 29.85% better than Cilium and 63.43% better than Istio. With its unique eBPF-based architecture, Cilium performs better than Istio and is a better choice for reducing overall resource utilization and operational costs cluster-wide. Further, as Cilium is both a CNI and a service mesh, its reduced resource requirements may suit edge devices where deploying a full-fledged service mesh is infeasible. With the absence of a sidecar, Cilium is also a suitable solution that can be offloaded to hardware accelerators such as DPU [21]. Our future work will focus on evaluating the impact of various CNIs on service mesh performance.

ACKNOWLEDGEMENTS

This material is based upon work supported by the National Science Foundation under Grant Numbers OAC-2346718 and OAC-2417607.

REFERENCES

- [1] Google and IBM. (2017) Istio Service Mesh. Accessed on August 15, 2024. [Online]. Available: <https://istio.io/>
- [2] Buoyant. (2016) Linkerd Service Mesh. Accessed on August 15, 2024. [Online]. Available: <https://linkerd.io/>
- [3] Hashicorp. (2018) Consul Service Mesh. Accessed on August 15, 2024. [Online]. Available: <https://www.consul.io/>
- [4] T. Labs, "Traefik - The Cloud Native Edge Router," 2024, accessed on August 15, 2024. [Online]. Available: <https://traefik.io/>
- [5] Isovalant. (2022) Cilium CNI. Accessed on August 15, 2024. [Online]. Available: <https://cilium.io/>
- [6] Google. (2018) Online Botique Microservices Demo. Accessed on August 15, 2024. [Online]. Available: <https://github.com/GoogleCloudPlatform/microservices-demo>
- [7] B. C. Gain. (2023) Will Istio or Linkerd Dominate Service Mesh? Accessed on August 15, 2024. [Online]. Available: <https://cloudnativenow.com/features/will-istio-or-linkerd-dominate-service-mesh/>
- [8] T. Asik and Y. E. Selcuk, "Policy enforcement upon software based on microservice architecture," in *2017 IEEE 15th International Conference on Software Engineering Research, Management and Applications (SERA)*, 2017, pp. 283–287.
- [9] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture," *IEEE Software*, vol. 33, no. 3, pp. 42–52, 2016.
- [10] F. Tapia, M. Á. Mora, W. Fuentes, H. Aules, E. Flores, and T. Toulkeridis, "From monolithic systems to microservices: A comparative study of performance," *Applied sciences*, vol. 10, no. 17, p. 5797, 2020.
- [11] D. B. Holenstein. (2015) A Modern Look at Reliability, Availability, and Scalability – Part 1. Accessed on August 15, 2024. [Online]. Available: <https://www.shadowbasesoftware.com/wp-content/uploads/2019/05/A-Modern-Look-at-Reliability-Availability-and-Scalability-Part-1.pdf>
- [12] B. Shafabakhsh, R. Lagerström, and S. Hacks, "Evaluating the Impact of Inter-Process Communication in Microservice Architectures," in *8th International Workshop on Quantitative Approaches to Software Quality*, 2020, pp. 55–63.
- [13] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, "Data management in microservices: State of the practice, challenges, and research directions," *arXiv preprint arXiv:2103.00170*, 2021.
- [14] T. Salah, M. J. Zemerly, C. Y. Yeun, M. Al-Qutayri, and Y. Al-Hammadi, "The evolution of distributed systems towards microservices architecture," in *2016 11th International Conference for Internet Technology and Secured Transactions (ICITST)*. IEEE, 2016, pp. 318–325.
- [15] N. Mateus-Coelho, M. Cruz-Cunha, and L. G. Ferreira, "Security in microservices architectures," *Procedia Computer Science*, vol. 181, pp. 1225–1236, 2021.
- [16] L. Bass, I. Weber, and L. Zhu, *DevOps: A software architect's perspective*. Addison-Wesley Professional, 2015.
- [17] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A Survey on Observability of Distributed Edge Container-Based Microservices," *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022.
- [18] Lyft. (2016) Envoy-Proxy. Accessed on August 15, 2024. [Online]. Available: <https://www.envoyproxy.io/>
- [19] K. Keahey, J. Anderson, Z. Zhen, P. Riteau *et al.*, "Lessons Learned from the Chameleon Testbed," in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.
- [20] J. Linäke. (2011) Locust. Accessed on August 15, 2024. [Online]. Available: <https://github.com/linkerd/linkerd2-proxy>
- [21] K. M. Hasanath, S. Basu, and D. Nadig, "Data Processing Unit (DPU) Based Network Process Offloading for Efficient Service Meshes," in *2024 IEEE 10th International Conference on Network Softwarization (NetSoft)*, 2024, pp. 319–321.